

普及组 CSP-J 2024 初赛模拟卷 6

扫码获取
答案解析

一、单项选择题（共 15 题，每题 2 分，共计 30 分；每题有且仅有一个正确选项）

- 2024 的因子与质因子分别有（ ）个。
A. 18 和 3 B. 16 和 3 C. 15 和 3 D. 16 和 4
- 使用邻接矩阵表示 N 个结点的有向图，所需要的存储空间为（ ）。
A. $N \times (N+1)$ B. N^2 C. $N \times (N-1)$ D. $N \times (N-1)/2$
- 在 C++ 程序中，表达式 $a\%=b$ 与下列哪个表达式是等价的？（ ）
A. $a=\%b$ B. $a/=b$ C. $a=b\%a$ D. $a=a\%b$
- 线性表若采用链表存储结构，则要求内存中可用存储单元地址（ ）。
A. 必须连续 B. 必须不连续
C. 连续或不连续都行 D. 部分连续
- 我们输入一个新闻网站的网址便可访问该网站，其中用到的网络协议是（ ）。
A. DNS B. FTP C. SSH D. TELNET
- 以下哪个不属于 STL 中栈的操作函数？（ ）
A. empty B. front C. push D. pop
- 平面上任取 n 个整点（横坐标和纵坐标都是整数），其中一定存在两个点，它们的中点也是整点，那么 n 至少是（ ）。
A. 4 B. 5 C. 6 D. 7
- 以下哪个操作属于位运算范畴？（ ）
A. $\&\&$ B. $||$ C. $>>>$ D. $\wedge$
- 关于树这种数据结构，下面的说法中哪个是正确的？（ ）
A. 满二叉树的结点总数一定是奇数

- B. 完全二叉树的结点总数一定是奇数
C. 树形结构只有双亲表示法和孩子表示法
D. 二叉树的遍历方法只有前序遍历法和后序遍历法
10. 以下哪个选项不属于头文件 `cmath`? ()
A. `find(iterator first, iterator last, int x)`
B. `abs(int x)`
C. `ceil(double x)`
D. `pow(double x, double y)`
11. 在 C++ 语言中, 表达式 `564|3` 的值等于 ()。
A. 7 B. 5 C. 4 D. 3
12. 定义变量 `double n`, 如果下面的代码输入为 1000, 则输出最接近 ()。

```
#include<bits/stdc++.h>
using namespace std;
int main()
{
    double n;
    cin>>n;
    cout << log10(n)-log2(n)<< endl;
    return 0;
}
```


A. 0 B. -5 C. -7 D. 7
13. 在图的广度优先搜索中, 要维护一个标识数组表示已经访问过的图的结点, 需要 () 数据结构存放结点以实现遍历。
A. 栈 B. 队列 C. 哈希表 D. 堆
14. 从一个 6×6 的棋盘 (不可旋转) 中选取不在同一行也不在同一列的两个方格, 共有 () 种方法。
A. 480 B. 450 C. 360 D. 720
15. 下列关于集合的说法哪个不正确? ()
A. 一个元素是否属于一个集合是确定的

- B. 集合中的元素两两不同
- C. 0 属于空集
- D. 集合中的元素不存在先后次序

二、阅读程序（程序输入不超过数组或字符串定义的范围；判断题正确填√，错误填×；除特殊说明外，判断题每题 1.5 分，选择题每题 3 分，共计 40 分）

(1)

```
01 #include <bits/stdc++.h>
02 using namespace std;
03 int main()
04 {
05     set <int> s;
06     srand(time(0));
07     int n=6;
08     int l=12;
09     int k,i=0,a[n];
10     while(s.size()<n)
11     {
12         k=rand()%10+1;
13         s.insert(k);
14     }
15     set <int>::iterator it;
16     for(it=s.begin(); it!=s.end(); it++)
17         a[i++] = *it;
18     for(int i=0; i<n; i++)
19     {
20         int x=rand()%n;
21         int y=rand()%n;
22         if(a[x] > a[y])
23             swap(a[x],a[y]);
24     }
25     for(int i=0; i<n; i++)
26         cout<<a[i]<<" ";
27     return 0;
28 }
```

■ 判断题

16. 若去掉第 6 行，程序的输出结果不受任何影响。

()

17. 若去掉第 8 行, 程序的输出结果中的数字总数不变。 ()
18. 程序的输出结果是一个从小到大排列的整数序列。 ()
19. 将第 10 行中的 `s.size()` 替换成 `s.length()`, 程序的运行结果不会改变。 ()

■ 选择题

20. 将第 13 行 `s.insert(k)` 替换成 `for(int i=1;i<=6;i++) s.insert(i)`, 则输出为 ()。
- A. 1 2 3 4 5 6 B. 6 5 4 3 2 1
- C. 1 2 3 6 5 4 D. 1~6 随机分布值
21. 将第 13 行 `s.insert(k)` 替换成 `for(int i=1;i<=7;i++) s.insert(i)`, 第 20 行替换为 `int x=i+1`, 第 21 行替换为 `int y=i`, 则输出为 ()。
- A. 1 2 3 4 5 6 B. 6 5 4 3 2 1
- C. 2 3 4 5 6 7 1 D. 2 3 4 5 6 7

(2)

```
01 #include<bits/stdc++.h>
02 using namespace std;
03 int month[13]={-1,31,28,31,30,31,30,31,31,30,31,30,31};
04 int date,ans1,ans2,y,m,d;
05 bool check1(int date){
06     char s[32];
07     sprintf(s, "%d", date);
08     if(s[0]==s[7] && s[1]==s[6] && s[2]==s[5] && s[3]==s[4])
09         return true;
10     return false;
11 }
12 bool check2(int date){
13     char s[32];
14     sprintf(s, "%d", date);
15     if(s[0]==s[2] && s[0]==s[5] && s[0]==s[7] && s[1]==s[3]
        && s[1]==s[4] && s[1]==s[6])
16         return true;
17     return false;
18 }
19 int main()
20 {
21     cin>>date;
```

```
22     y=date/10000;
23     m=date/100%100;
24     d=date%100;
25     for(int i=y; ;i++){
26         if(i%400==0 || (i%100!=0 && i%4==0))
27             month[2] = 29;
28         else
29             month[2] = 28;
30     int j=(i==y)?m:1;
31     for( ;j<=12;j++){
32         int k=(i==y && j==m)?d+1:1;
33         for(;k<=month[j];k++){
34             int date=i*10000+j*100+k;
35             if(check1(date) && ans1==0)
36                 ans1=date;
37             if(check2(date))
38                 return cout<<ans1<<" "<<date<<endl,0;
39         }
40     }
41 }
42 return 0;
43 }
```

注：输入为 8 位数字。

■ 判断题

22. 将第 3 行中的 -1 改为 0，程序的运行不受任何影响。 ()
23. 去掉第 15 行中的 `&& s[1]==s[4] && s[1]==s[6]`，程序的输出不变。 ()
24. 将第 23 行 `m=date/100%100` 替换为 `m=date%10000/100`，程序的输出不变。 ()
25. 将第 30 行改为 `int j; if(i==y && j==1 || i!=y && j==0) j=m`，程序的输出不变。 ()

■ 选择题

26. 若输入 20240204，则输出为 ()。
- A. 20240204 20300302 B. 20300302 20400402
- C. 20400402 21211212 D. 20300302 21211212

27. 若输出 20011002 20200202, 则输入可能为 ()。

- A. 20011002 B. 20001001 C. 20020101 D. 20020202

(3)

```
01 #include<bits/stdc++.h>
02 using namespace std;
03 const int MAXN = 100;
04 bool Prime_Number_Judge(const int &num){
05     if (num <= 3)
06         return num > 1;
07     for (int i = 2; i < num; i++)
08         if (num % i == 0)
09             return false;
10     return true;
11 }
12 int& Get_Number_Size(const int &num){
13     int digit = 0, val = num;
14     while (val){
15         val /= 10;
16         digit++;
17     }
18     return digit;
19 }
20 vector<int>&Get_Digits(const int &num, vector<int> &digits){
21     int vactor_val = 0;
22     for(int num_size=Get_Number_Size(num);num_size>0;num_size--){
23         vactor_val = num % (int)pow(10.0, num_size);
24         vactor_val = vactor_val/((int)pow(10.0,num_size-1));
25         digits.push_back(vactor_val);
26     }
27     return digits;
28 }
29 vector<int>&Get_K_Adjacent(const int &num, vector<int>&adjacent){
30     vector<int> digists_number;
31     char tmp[128], buf_tmp[128];
32     Get_Digits(num , digists_number);
33     int digits = Get_Number_Size(num);
34     for (int i = 0; i < digits; i++){
35         for (int j = 0; j < digits-i; j++){
36             string buf;
```

```
37         int k = 0;
38         while (k <= i){
39             sprintf(tmp,"%d",digists_number.at(j+k));
40             buf += tmp;
41             k++;
42         }
43         for(int i=0;i<=buf.size();i++)
44             buf_tmp[i] = buf[i];
45         adjacent.push_back(atoi(buf_tmp));
46     }
47 }
48 return adjacent;
49 }
50 int main(){
51     int count=0;
52     for (int i = 1; i < MAXN; i++){
53         if (Prime_Number_Judge(i)){
54             vector<int> buf;
55             Get_K_Adjacent(i, buf);
56             int sign = 1;
57             for (int j = 0; j < buf.size(); j++){
58                 if (!Prime_Number_Judge(buf.at(j))){
59                     sign = 0;
60                     break;
61                 }
62             }
63             if (sign)
64                 count++;
65         }
66     }
67     cout<<count;
68     return 0;
69 }
```

■ 判断题

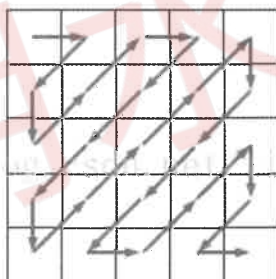
28. 若将第 5 行 `if (num <= 3)` 替换为 `if (num < 3)`, 程序的运行结果不会改变。 ()
29. 若将第 7 行中的 `i < num` 替换为 `i*i <= num`, 程序的运行结果不会改变。 ()
30. 本程序用到的 `vector` 属于 STL。 ()
31. 若将 `MAXN=100` 替换为 `MAXN=70`, 程序的运行结果不会改变。 ()

■ 选择题

32. 运行本程序，输出结果为 ()。
- A. 7 B. 8 C. 9 D. 10
33. 若将 $\text{MAXN} = 100$ 改为 $\text{MAXN} = 2024$ ，程序的输出结果为 ()。
- A. 9 B. 10 C. 11 D. 12
34. (4 分) 若将 $\text{MAXN} = 100$ 改为 $\text{MAXN} = 20244202$ ，程序的输出结果为 ()。
- A. 11 B. 10 C. 9 D. 大于 11 的整数

三、完善程序 (单选题，每小题 3 分，共计 30 分)

- (1) 在图像编码的算法中，需要对一个给定的方形矩阵进行 Z 字形扫描。给定一个 $n \times n$ 的矩阵，Z 字形扫描的过程如下图所示。



对 4×4 的矩阵

1 5 3 9

3 7 5 6

9 4 6 4

7 3 1 3

进行 Z 字形扫描后得到长度为 16 的序列：

1 5 3 9 7 3 9 5 4 7 3 6 6 4 1 3

请实现一个 Z 字形扫描的程序。给定一个 $n \times n$ 的矩阵，输出对这个矩阵进行 Z 字形扫描的结果。

输入格式：

输入的第 1 行包含一个整数 n ，表示矩阵的大小。输入的第 2 行到第 $n+1$ 行每行包含 n 个正整数，由空格分隔，表示给定的矩阵。

输出格式：

输出一行，包含 $n \times n$ 个整数，由空格分隔，表示输入的矩阵经过 Z 字形扫描后的结果。

输入样例：

```
4
1 5 3 9
3 7 5 6
9 4 6 4
7 3 1 3
```

输出样例：

```
1 5 3 9 7 3 9 5 4 7 3 6 6 4 1 3
```

评测用例规模与约定：

$1 \leq n \leq 500$ ，矩阵元素为不超过 1000 的正整数。

```
01 #include<bits/stdc++.h>
02 using namespace std;
03 int main()
04 {
05     int n,t,x,y;
06     t=1, x=1, y=1;
07     int v[505][505], p[505][505];
08     cin >> n;
09     for(int i=1;i<=n;i++)
10         for(int j=1;j<=n;j++)
11             cin>>p[i][j];
12     printf("%d ", ①);
13     ②;
14     while(③)
15     {
16         if(y+1 <= n)
17         {
18             ++t;
19             y++;
20             v[x][y] = 1;
21             printf("%d ",p[x][y]);
22         }
23         while(x+1<=n && y-1>=1 && ④)
24         {
```

```
25         ++t;
26         v[x+1][y-1] = 1;
27         printf("%d ",p[x+1][y-1]);
28         x++;
29         y--;
30     }
31     if(x+1<=n)
32     {
33         ++t;
34         x++;
35         v[x][y] = 1;
36         printf("%d ",p[x][y]);
37     }
38     while(x-1 >= 1 && y+1 <= n && !v[x-1][y+1])
39     {
40         ++t;
41         ⑤;
42         printf("%d ",p[x-1][y+1]);
43         x--;
44         y++;
45     }
46 }
47 return 0;
48 }
```

35. ①处应填 ()。

- A. p[0][0] B. p[0][1] C. p[1][0] D. p[1][1]

36. ②处应填 ()。

- A. v[0][0] = 1 B. v[1][1] = 1 C. v[0][1] = 1 D. v[1][0] = 1

37. ③处应填 ()。

- A. t < n*n B. t <= n*n C. t < n D. t <= n

38. ④处应填 ()。

- A. v[x+1][y-1] B. v[x-1][y+1]
C. !v[x+1][y-1] D. !v[x-1][y+1]

39. ⑤处应填 ()。

A. $v[x-1][y-1] = 1$

B. $v[x+1][y+1] = 1$

C. $v[x+1][y-1] = 1$

D. $v[x-1][y+1] = 1$

(2) 你有一架天平和 N 个砝码, 这 N 个砝码的重量依次是 $w_1, w_2, \dots, w_N$ 。请计算: 一共可以称出多少种不同的重量? 注意砝码可以放在天平两边。

输入格式:

输入的第 1 行包含一个整数 N 。第 2 行包含 N 个整数: $w_1, w_2, \dots, w_N$ 。

输出格式:

输出一个整数代表答案。

输入样例:

3

1 4 6

输出样例:

10

样例说明:

能称出的 10 种重量是 1、2、3、4、5、6、7、9、10、11。

1=1

2=6-4 (天平一边放 6, 另一边放 4)

3=4-1

4=4

5=6-1

6=6

7=1+6

9=4+6-1

10=4+6

11=1+4+6

评测用例规模与约定:

对于 50% 的评测用例, $1 \leq N \leq 15$;

对于所有评测用例, $1 \leq N \leq 100$, N 个砝码的总重量不超过 10^5 。

```
01 #include <bits/stdc++.h>
02 using namespace std;
03 const int maxn = 105, maxv = 1e5 + 5;
```

```
04 int n, a[maxn], f[maxn][maxv], sum, ans;
05 int main()
06 {
07     cin >> n;
08     for(int i = 1; i <= n; i++)
09     {
10         cin >> a[i];
11         ①;
12     }
13     f[0][0] = 1;
14     for(int i = 1; i <= n; i++)
15     {
16         for(int ②; j >= 0; j--)//2
17         {
18             f[i][j] |= f[i - 1][j];
19             f[i][j] |= ③;
20             if( ④ )
21                 f[i][j] |= f[i - 1][j + a[i]];
22         }
23     }
24     for(int i = 1; i <= sum; i++)
25         ⑤;
26     cout << ans;
27     return 0;
28 }
```

40. ①处应填 ()。

A. `sum += a[i]`

B. `sum += a[1]`

C. `sum += a[n]`

D. `sum = a[i]`

41. ②处应填 ()。

A. `j = f[i][j]`

B. `j = sum-1`

C. `j = n`

D. `j = sum`

42. ③处应填 ()。

A. `f[i-1][j-a[i]]`

B. `f[i-1][abs(j-a[i])]`

C. `f[i][abs(j-a[i])]`

D. `f[i-1][a[i]-j]`

43. ④处应填 ()。

A. $j + a[i] < \text{sum}$

B. $j + a[i] \leq \text{sum}$

C. $j + a[i-1] < \text{sum}$

D. $j + a[i-1] \leq \text{sum}$

44. ⑤处应填 ()。

A. $\text{ans} += f[n][i-1]$

B. $\text{ans} += f[n-1][i-1]$

C. $\text{ans} += f[n][i]$

D. $\text{ans} += f[n-1][i]$

试用水印



扫码获取
答案解析

普及组 CSP-J 2024 初赛模拟卷 7

一、单项选择题（共 15 题，每题 2 分，共计 30 分；每题有且仅有一个正确选项）

1. 在标准 ASCII 码表中，字符'4'的 ASCII 码值用二进制表示是（ ）。
A. 00000100 B. 00110100 C. 00110101 D. 00110011
2. 关于树这种数据结构的下述说法，正确的是（ ）。
A. 一个有 m 个顶点和 $m-1$ 条边的图就是树
B. 树中的任意两个顶点之间有且只有一条简单路径
C. 树中有的结点可能构成环
D. 若树根层次为 1，则对应高度为 n 的二叉树最多有 2^n 个结点
3. 以下哪个不是输入设备？（ ）
A. 绘图仪 B. 触摸屏 C. 扫描仪 D. 麦克风
4. 当 $a=3$, $b=2$, $c=1$ 时，执行以下程序段后 $c=$ （ ）。

```
if(a>b)
    a=b;
if(b>c)
    b=c;
else
    c=b;
c=a--;
```


A. 0 B. 1 C. 2 D. 3
5. 学生在大学选修某些课程时需要先上其他的前置课程，所有课程和课程间的先修关系构成一个有向图 G ，有向边 $\langle M, N \rangle$ 表示课程 M 是课程 N 的先修课，则要找到某门课程 L 的全部先修课，下面哪种算法不可行？（ ）
A. Dijkstra B. BFS C. DFS D. BFS+DFS
6. 下列对语句 `freopen("function.in", "r", stdin);` 的分析中正确的是（ ）。

- A. freopen 是文件名
B. function.in 是重定向函数名
C. r 代表重定向为“写”方式
D. 语句将 cin 重定向到文件 function.in
7. Windows 下可执行文件的扩展名是 ()。
A. com B. exe C. cpp D. dll
8. [x]补码=10011000, 其原码为 ()。
A. 011001111 B. 11101000 C. 11100110 D. 01100101
9. 下面有关布尔类型的函数的说法, 正确的是 ()。
A. 布尔类型函数只能返回 0 和 1 两个值
B. 布尔类型函数可以返回负数
C. 布尔类型函数必须有参数传递
D. 布尔类型函数可以返回 NULL
10. 下面有关格雷码的说法, 错误的是 ()。
A. 在格雷码中, 任意两个相邻的代码只有一位二进制数不同
B. 格雷码是一种可靠性编码
C. 在格雷码中, 最大数和最小数只有一位二进制数不同
D. 格雷码是一种唯一性编码
11. 现在有 5 个整数 -2, -1, 0, 1, 2, 从中任意挑选两个整数, 它们的和为 0 的概率是多少?
()
A. 1/6 B. 1/4 C. 1/5 D. 1/10
12. 小明走楼梯, 每次上台阶能跨 1 或 2 级。下面是走到第 N 步台阶的 C++ 实现代码。
该段代码采用的是 () 算法。
- ```
int UpStairs(int N)
{
 if(N==1)
 return 1;
 else if(N==2)
 return 2;
```

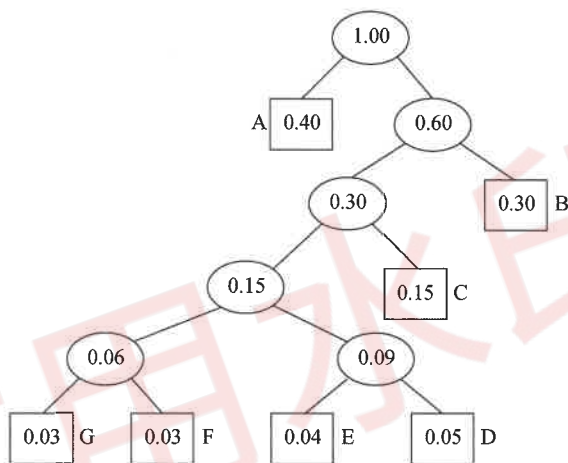
```

else
 return UpStairs(N-2) + UpStairs(N-1);
}

```

A. 递推                      B. 贪心                      C. 动态规划                      D. 分治

13. 某内容中仅会出现 A, B, C, D, E, F, G, 对应的出现概率分别为 0.40, 0.30, 0.15, 0.05, 0.04, 0.03, 0.03, 如下图所示。按照哈夫曼编码规则, 假设 B 的编码为 11, 则 D 的编码为 ( )。



A. 10010                      B. 10011                      C. 10111                      D. 10001

14. 某学习小组有 5 名男生和 3 名女生, 从中选 3 名男生和 1 名女生参加 3 项竞赛活动, 每项活动至少有 1 人参加, 则参赛方法有 ( ) 种。

A. 960                      B. 1080                      C. 2160                      D. 540

15. 简单无向连通图 G 有 18 条边, 且每个顶点的度数为 2, 则图 G 有 ( ) 个顶点。

A. 81                      B. 17                      C. 18                      D. 64

二、阅读程序 (程序输入不超过数组或字符串定义的范围; 判断题正确填√, 错误填×; 除特殊说明外, 判断题每题 1.5 分, 选择题每题 3 分, 共计 40 分)

(1)

```

01 #include<bits/stdc++.h>
02 using namespace std;

```

```
03 int a[101000];
04 int p(int deep){
05 return pow(2,deep);
06 }
07 int main(){
08 int n,mindeep=1,i=2,deep=2;
09 scanf("%d %d",&n,&a[1]);
10 long long maxx = a[1];
11 for(i=2;i<=n;i++)
12 scanf("%d",&a[i]);
13 i=2;
14 while(i<=n){
15 long long sum=0;
16 for(;i<p(deep)&&i<=n;i++)
17 sum+=a[i];
18 if(sum > maxx)
19 maxx = sum, mindeep = deep;
20 deep++;
21 }
22 cout << mindeep;
23 return 0;
24 }
```

### ■ 判断题

16. 将第 8 行中的 `i=2` 改为 `i`, 程序的运行结果不会改变。 ( )
17. 如果输入都是正整数, 那么将第 10 行中的 `a[1]` 替换成 `a[0]`, 将第 13 行中的 `i=2` 替换成 `i=1`, 程序的运行结果不会改变。 ( )
18. 将第 15 行中的 `long long sum=0;` 替换成 `long long sum;`, 程序的运行结果不会改变。 ( )
19. 将第 18 行中的 `if(sum > maxx)` 替换成 `if(sum >= maxx)`, 程序的运行结果不会改变。 ( )

### ■ 选择题

20. 若输入为 7 1 6 5 4 3 2 1, 则输出为 ( )。
- A. 11                      B. 2                      C. 10                      D. 3
21. 若输入为 15 9 4 5 3 3 2 1 2 1 1 1 1 1 1, 则输出为 ( )。
- A. 4                      B. 3                      C. 2                      D. 1

## (2)

```
01 #include <bits/stdc++.h>
02 using namespace std;
03 bool judge(int a)
04 {
05 while(a!=0)
06 {
07 int t = a%10;
08 if(t==2 || t==4)
09 return 0;
10 a = a/10;
11 }
12 return 1;
13 }
14 int main(void)
15 {
16 int sum=0,n;
17 cin>>n;
18 for(int i=1;i<n/3+1;i++)
19 if(judge(i))
20 for(int j=i+1;j<n-i-j;j++)
21 if(judge(j) && judge(n-i-j))
22 {
23 sum++;
24 }
25 cout<<sum;
26 return 0;
27 }
```

## ■ 判断题

22. 将 judge 函数和 main 函数的位置调换一下, 程序的运行不受影响。 ( )
23. 将第 3 行中的 bool 改为 int, 程序的运行结果不会改变。 ( )
24. 该程序运行的时间复杂度是  $O(n\log n)$ 。 ( )
25. 将第 20 行中的  $j=i+1$  改为  $j=1$ , 程序的运行结果不会改变。 ( )

## ■ 选择题

26. 若输入  $n = 24$ , 则程序的输出为 ( )。
- A. 15                      B. 16                      C. 14                      D. 13

27. 若程序的输出为 20, 则程序的输入可能为 ( )。

A. 38

B. 41

C. 40

D. 42

(3)

```
01 #include<bits/stdc++.h>
02 using namespace std;
03 typedef long long LL;
04 int a[100005],b[100005],c[100005];
05 int small(int n, int a[], int key)
06 {
07 int left=1, right=n;
08 while(left < right)
09 {
10 int mid = (left+right+1)/2;
11 if(a[mid] < key)
12 left = mid;
13 else
14 right = mid - 1;
15 }
16 return left;
17 }
18 int big(int n, int c[], int key)
19 {
20 int left=1, right=n;
21 while(left < right)
22 {
23 int mid = (left+right)/2;
24 if(c[mid] > key)
25 right = mid;
26 else
27 left = mid + 1;
28 }
29 return left;
30 }
31 int main()
32 {
33 int n;
34 cin>>n;
35 for(int i=1;i<=n;++i)
36 cin>>a[i];
```



34. (4 分) 若输入 3 1 2 3 3 4 5 5 6 7, 那么输出结果是 ( )。

A. 27

B. 21

C. 18

D. 15

### 三、完善程序 (单选题, 每小题 3 分, 共计 30 分)

(1) 一个  $n \times n$  的正方形的黑白瓦片的图案要被转换成新的正方形图案。写一个程序来找出将原始图案按照下列转换方法转换成新图案所采用的最小序号的方式。

①转  $90^\circ$ : 图案顺时针旋转  $90^\circ$ 。

②转  $180^\circ$ : 图案顺时针旋转  $180^\circ$ 。

③转  $270^\circ$ : 图案顺时针旋转  $270^\circ$ 。

④反射: 图案在水平方向翻转 (以中央铅垂线为中心形成原图案的镜像)。

⑤组合: 图案在水平方向翻转, 然后再按照①~③中的一种再次转换。

⑥不改变: 原图案不改变。

⑦无效转换: 无法用以上方法得到新图案。

如果有多种可用的转换方法, 请选择序号最小的那个。

只使用上述 7 种步骤中的一个来完成这次转换。

输入格式:

第 1 行一个正整数  $n$ 。然后是  $n$  行, 每行  $n$  个字符, 全部为 @ 或 -, 表示初始的正方形。接下来的  $n$  行, 每行  $n$  个字符, 全部为 @ 或 -, 表示最终的正方形。

输出格式:

单独的一行包括 1~7 中的一个数字 (在上文已描述), 表明需要将转换前的正方形变为转换后的正方形的转换方法。

输入样例:

```
3
@-@

@@@
@-@
@--
--@
```

输出样例:

```
1
```

数据范围:

对于 100% 的数据,  $1 \leq n \leq 10$ 。

```
01 #include<bits/stdc++.h>
02 using namespace std;
03 int n;
04 char a[15][15],b[15][15],c[15][15],d[15][15];
05 bool work1(){
06 for(int i=1;i<=n;i++)
07 for(int j=1;j<=n;j++)
08 ①;
09 for(int i=1;i<=n;i++)
10 for(int j=1;j<=n;j++)
11 if(b[i][j]!=c[i][j]) return 0;
12 return 1;
13 }
14 bool work2(){
15 for(int i=1;i<=n;i++)
16 for(int j=1;j<=n;j++)
17 ②;
18 for(int i=1;i<=n;i++)
19 for(int j=1;j<=n;j++)
20 if(b[i][j]!=c[i][j]) return 0;
21 return 1;
22 }
23 bool work3(){
24 for(int i=1;i<=n;i++)
25 for(int j=1;j<=n;j++)
26 ③;
27 for(int i=1;i<=n;i++)
28 for(int j=1;j<=n;j++)
29 if(b[i][j]!=c[i][j]) return 0;
30 return 1;
31 }
32 bool work4(){
33 for(int i=1;i<=n;i++)
34 for(int j=1;j<=n;j++)
35 ④;
36 for(int i=1;i<=n;i++)
37 for(int j=1;j<=n;j++)
38 if(b[i][j]!=c[i][j]) return 0;
39 return 1;
40 }
```

```
41 bool work5(){
42 ⑤;
43 for(int i=1;i<=n;i++)
44 for(int j=1;j<=n;j++) a[i][j]=b[i][j];
45 if(work1()) return 1;
46 for(int i=1;i<=n;i++)
47 for(int j=1;j<=n;j++) a[i][j]=b[i][j];
48 if(work2()) return 1;
49 for(int i=1;i<=n;i++)
50 for(int j=1;j<=n;j++) a[i][j]=b[i][j];
51 if(work3()) return 1;
52 return 0;
53 }
54 bool work6(){
55 for(int i=1;i<=n;i++)
56 for(int j=1;j<=n;j++)
57 if(b[i][j]!=c[i][j]) return 0;
58 return 1;
59 }
60 void work(){
61 if(work1()) {cout<<1; return;}
62 if(work2()) {cout<<2; return;}
63 if(work3()) {cout<<3; return;}
64 if(work4()) {cout<<4; return;}
65 if(work5()) {cout<<5; return;}
66 if(work6()) {cout<<6; return;}
67 cout<<7;
68 }
69 int main(){
70 cin>>n;
71 for(int i=1;i<=n;i++)
72 for(int j=1;j<=n;j++) {
73 cin>>a[i][j];
74 d[i][j]=a[i][j];
75 }
76 for(int i=1;i<=n;i++)
77 for(int j=1;j<=n;j++) cin>>c[i][j];
78 work();
79 return 0;
80 }
```

35. ①处应填 ( )。

A.  $b[j][n-i+1]=a[j][i]$

B.  $b[j][n-i]=a[j][i]$

C.  $b[j][n-i+1]=a[i][j]$

D.  $b[j][n-i]=a[i][j]$

36. ②处应填 ( )。

A.  $b[n-i][n-j]=a[i][j]$

B.  $b[n-i+1][n-j+1]=a[i][j]$

C.  $b[n-i][n-j]=a[j][i]$

D.  $b[n-j+1][n-i+1]=a[i][j]$

37. ③处应填 ( )。

A.  $b[n-j+1][i]=a[i][j]$

B.  $b[n-j][i]=a[i][j]$

C.  $b[n-j+1][i]=a[j][i]$

D.  $b[n-j][i]=a[j][i]$

38. ④处应填 ( )。

A.  $b[n-j+1][i]=a[i][j]$

B.  $b[i][n-j]=a[i][j]$

C.  $b[i][n-j+1]=a[i][j]$

D.  $b[i][n-j]=a[j][i]$

39. ⑤处应填 ( )。

A. `work1()`

B. `work2()`

C. `work3()`

D. `work4()`

(2) 一个旅行家想驾驶汽车以最少的费用从一个城市到另一个城市 (假设出发时油箱是空的)。给定两个城市之间的距离  $D_1$ 、汽车油箱的容量  $C$  (以升为单位)、每升汽油能行驶的距离  $D_2$ 、出发点每升汽油的价格  $P$ 、沿途加油站数  $N$  ( $N$  可以为零)、加油站  $i$  离出发点的距离  $D_i$ 、每升汽油的价格  $P_i$  ( $i=1,2,\dots,N$ )。计算结果四舍五入至小数点后两位。如果无法到达目的地, 则输出 No Solution。

输入格式:

第 1 行,  $D_1, C, D_2, P, N$ 。接下来有  $N$  行。第  $i+1$  行, 两个数字, 即加油站  $i$  离出发点的距离  $D_i$  和每升汽油的价格  $P_i$ 。

输出格式:

所需的最少费用, 计算结果四舍五入至小数点后两位。如果无法到达目的地, 则输出 No Solution。

输入样例:

275.6 11.9 27.4 2.8 2

102.0 2.9

220.0 2.2

输出样例:

26.95

说明/提示:

$N \leq 6$ , 其余数字  $\leq 500$ 。

```
01 #include<bits/stdc++.h>
02 using namespace std;
03 const int MAXN = 100;
04 int N;
05 double D[MAXN], P[MAXN], D1, C, per, req;
06 int main(){
07 cin>>D1>>C>>per>>P[0]>>N;
08 D1 /= per;
09 for(int i=1; i<=N; i++) {
10 cin>>D[i]>>P[i];
11 D[i] /= per;
12 }
13 D[0] = 0;
14 ①;
15 P[N+1] = 0;
16 double cur = 0, ans = 0;
17 for(int i=0; i<=N; i++) {
18 int ni = N+1;
19 for(int j=i+1; j<= N+1; j++) {
20 if(P[j] <= P[i]) {
21 ni = j;
22 break;
23 }
24 }
25 ②;
26 if(req > C)
27 ③;
28 if(req > cur) {
29 ans += (req - cur) * P[i];
30 cur = req;
31 }
32 ④;
33 if(cur < 0)
34 break;
```

```
35 }
36 if(cur < 0)
37 cout<<"No Solution"<<endl;
38 else
39 ⑤;
40 return 0;
41 }
```

40. ①处应填 ( )。

A.  $D[N] = D1$

B.  $D[N+1] = D1$

C.  $D[1] = D1$

D.  $D[N-1] = D1$

41. ②处应填 ( )。

A.  $req = D[i] - D[i-1]$

B.  $req = D[i+1] - D[i]$

C.  $req = D[ni] + D[i]$

D.  $req = D[ni] - D[i]$

42. ③处应填 ( )。

A.  $req = C$

B.  $req -= C$

C.  $req += C$

D.  $req \% = C$

43. ④处应填 ( )。

A.  $cur += D[i+1] - D[i]$

B.  $cur -= D[i+1] - D[i]$

C.  $cur += D[ni] - D[i]$

D.  $cur -= D[ni] - D[i]$

44. ⑤处应填 ( )。

A. `printf("%.2d\n",ans)`

B. `printf("%2lf\n",ans)`

C. `cout<<fixed<<setprecision(2)<<ans<<endl`

D. `cout<<fixed<<setprecision(3)<<ans<<endl`

## 普及组 CSP-J 2024 初赛模拟卷 8

扫码获取  
答案解析

## 一、单项选择题（共 15 题，每题 2 分，共计 30 分；每题有且仅有一个正确选项）

1. 在 C++ 语言中，以下哪种方式不能用于向函数传递参数？（ ）。  
A. 值传递      B. 模板传递      C. 引用传递      D. 指针传递
2. 以下关于算法的描述中正确的是（ ）。  
A. 算法是为解决问题而编制的计算机程序  
B. 算法是为解决问题而采用的计算机程序设计语言  
C. 算法是为解决问题而采用的计算方法  
D. 算法是为解决问题而采取的方法与步骤
3. 在关系数据库中，存放在数据库中的数据逻辑结构以（ ）为主。  
A. 二维表      B. 二叉树      C. 哈希表      D. 邻接表
4. 设某算法的时间复杂度函数的递推方程是  $T(n) = T(n-1) + n$  ( $n$  为正整数) 及  $T(0) = 1$ ，则该算法的时间复杂度为（ ）。  
A.  $O(\log n)$       B.  $O(n)$       C.  $O(n^2)$       D.  $O(n \log n)$
5. 在 C++ 语言中，一般提到的“空间复杂度”中的“空间”是指（ ）。  
A. 程序运行时所占的内存大小      B. 程序运行时所占的数组大小  
C. 程序运行时所占的硬盘大小      D. 程序源文件所占的硬盘大小
6. 2017 年 9 月 30 日是星期六，1999 年 9 月 30 日是（ ）。  
A. 星期三      B. 星期六      C. 星期五      D. 星期四
7. 设栈 S 的初始状态为空，元素 1, 2, 3, 4, 5 依次入栈，以下出栈序列中不可能出现的是（ ）。  
A. 1, 2, 3, 5, 4      B. 2, 3, 1, 5, 4      C. 1, 5, 3, 2, 4      D. 4, 3, 5, 2, 1
8. 关于分治算法，下列说法中错误的是（ ）。

- A. 分治算法的核心思想是分而治之，把问题转化为多个规模更小的子问题再求解
- B. 分治算法可以不使用递归实现
- C. 分治算法的时间复杂度是  $O(\log n)$ ，其中  $n$  表示问题的规模
- D. 二分法、快速排序等算法都是使用分治思想的典型算法

9. 如下代码主要表示什么数据结构？（ ）

```
typedef int LTDataType;
typedef struct ListNode
{
 struct ListNode* prev;
 struct ListNode* next;
 LTDataType data;
} LTNode;
```

- A. 单向链表      B. 双向链表      C. 循环链表      D. 优先队列

10. 使用如下代码片段定义 4 个字符串（假设头文件已正确定义），以下说法中错误的是（ ）。

```
string str1 = "csp";
string str2 = str1;
char str3[] = "csp";
char * str4 = str3;
```

- A. 对于 4 个字符串，都可以使用 `std::cout` 输出其中的内容（例如，`cout<<str3;`）
- B. `str3` 只占用 4 字节内存，但 `str1` 要占用更多内存
- C. 由于 `str2` 由 `str1` 直接赋值得到，因此二者指向同一块内存，即修改 `str1` 的内容后 `str2` 的内容也会随之改变
- D. 由于 `str4` 由 `str3` 直接赋值得到，因此二者指向同一块内存，即修改 `str3` 的内容后 `str4` 的内容也会随之改变

11. 冯·诺依曼体系结构中组成计算机的五大部件是（ ）。

- A. 总线、处理器、存储器、输入设备、输出设备
- B. 处理器、运算器、存储器、输入设备、输出设备
- C. 总线、控制器、存储器、输入设备、输出设备
- D. 控制器、运算器、存储器、输入设备、输出设备

12. 如下代码对树的操作是 ( )。

```
void postorder(tree bt)
{
 if(bt)
 {
 postorder(bt->lchild);
 postorder(bt->rchild);
 cout << bt->data;
 }
}
```

- A. 后序遍历      B. 中序遍历      C. 前序遍历      D. 层次遍历
13. 有规格尺寸相同的 5 种颜色的手套 (不分左右) 各 15 只混装在箱子里, 从中至少取出多少只, 才能保证配出 3 副颜色相同的手套? ( )
- A. 8      B. 9      C. 10      D. 11
14. 由 3 个 a、5 个 b 和 2 个 c 构成的字符串中, 包含子串 "abc" 的共有 ( ) 个。
- A. 39 600      B. 780      C. 840      D. 260
15. 1 GB 代表的字节数量是 ( )。
- A.  $2^{10}$       B.  $2^{20}$       C.  $2^{30}$       D.  $2^{40}$

二、阅读程序 (程序输入不超过数组或字符串定义的范围; 判断题正确填√, 错误填×; 除特殊说明外, 判断题每题 1.5 分, 选择题每题 3 分, 共计 40 分)

(1)

```
01 #include<bits/stdc++.h>
02 using namespace std;
03 const int SIZE = 25;
04 int n,k,a[SIZE];
05 int cnt=0;
06 long long ans;
07 bool isPrime(int n)
08 {
09 cnt++;
10 cout<<cnt<<endl;
```

```
11 if(n<=1)
12 return false;
13 for(int i=2;i*i<=n;++i)
14 if(0==n%i)
15 return false;
16 return true;
17 }
18 void dfs(int m, int sum, int x)
19 {
20 if(m==k)
21 {
22 if(isPrime(sum))
23 {
24 ans++;
25 }
26 return;
27 }
28 for(int i=x;i<n;++i)
29 dfs(m+1,sum+a[i],i+1);
30 return;
31 }
32 int main()
33 {
34 cin >> n >> k;
35 for (int i = 0; i < n;++i)
36 cin >> a[i];
37 dfs(0,0,0);
38 cout << ans <<endl;
39 return 0;
40 }
```

#### ■ 判断题

16. 将第 13 行中的  $i*i \leq n$  改为  $i \leq \sqrt{n}$ , 程序的运行结果不会改变。 ( )
17. 将第 14 行中的  $if(0==n\%i)$  改为  $if(n\%i==0)$ , 程序的运行结果不会改变。 ( )
18. 若输入  $k$  值比  $n$  大, 则程序会死循环, 无输出。 ( )
19. 将第 29 行中的  $sum+a[i]$  改为  $sum+=a[i]$ , 程序的运行结果不会改变。 ( )

## ■ 选择题

20. 若输入  $k$  的值为 6, 且输入  $n \geq k$ ,  $a$  数组的元素均为相等的正整数, 则输出为 ( )。
- A. 0                      B. 2                      C. 4                      D. 8
21. 若输入 12 8, 则 `isPrime` 被调用的次数为 ( )。
- A. 4                      B. 66                      C. 132                      D. 495

## (2)

```
01 #include <bits/stdc++.h>
02 using namespace std;
03 int CNT, p;
04 int check(int tmp, int n)
05 {
06 int res = 0;
07 for(int i=tmp; i>0; i--)
08 {
09 res += n;
10 int a=0, ans=res;
11 while(ans%n == 0)
12 a++, ans/=n;
13 i -= a-1;
14 }
15 return res;
16 }
17 int main()
18 {
19 cin >> CNT;
20 while(CNT--)
21 {
22 int ans=1;
23 cin >> p;
24 for(int i=2; i<=sqrt(p); i++)
25 {
26 int tmp=0;
27 while(p%i == 0)
28 tmp++, p/=i;
29 if(tmp)
30 ans = max(ans, check(tmp, i));
31 }
```

```
32 ans = max(ans,p);
33 cout<<ans<<endl;
34 }
35 return 0;
36 }
```

### ■ 判断题

22. 若输入 1 2, 程序的输出会报错。 ( )
23. 若将第 27 行中的 while 改为 if, 程序的输出结果不变。 ( )
24. 若输入 p 是一个素数, 则程序中的第 30 行不会被执行。 ( )
25. 若输入 p 是一个素数, 则输出 ans 的值为 p。 ( )

### ■ 选择题

26. 若输入 1 36, 则输出为 ( )。
- A. 36                      B. 6                      C. 4                      D. 9
27. 若  $\text{tmp} < n$ , 则返回的 res 结果为 ( )。
- A. n                      B. tmp                      C.  $\text{tmp} * n$                       D.  $\text{tmp} + n$

### (3)

```
01 #include<iostream>
02 #include<string>
03 #include<set>
04 using namespace std;
05 string a, s[105];
06 int cnt=0;
07 set<string>dict;
08 void trim(string &s)
09 {
10 if(!s.empty())
11 {
12 s.erase(0,s.find_first_not_of(" "));
13 s.erase(s.find_last_not_of(" ")+1);
14 }
15 }
16 int main()
17 {
```

```
18 cin>>a;
19 while(a!="#")
20 {
21 int length = a.length();
22 for(int i=0;i<length;i++)
23 if(isalpha(a[i]))
24 a[i] = tolower(a[i]);
25 else
26 a[i] = ' ';
27 trim(a);
28 s[cnt]=a;
29 dict.insert(a);
30 cnt++;
31 cin>>a;
32 }
33 for(set<string>::iterator it=dict.begin();it!=dict.end();++it)
34 cout << *it << " ";
35 return 0;
36 }
```

#### ■ 判断题

28. STL 中的 set 属于关联式容器。 ( )
29. 若将第 6 行中的 int cnt=0 替换为 int cnt, 程序的运行结果不会改变。 ( )
30. 若将第 21 行中的 a.length() 替换为 a.size(), 程序的运行结果不会改变。 ( )
31. 若输入 34 12 56 #, 则程序输出 12 34 56。 ( )

#### ■ 选择题

32. 若输入为 A Dream Is To Build A Big Big World #, 则输出是 ( )。
- A. A Dream Is To Build A Big Big World
- B. a dream is to build a big big world
- C. a big build dream is to world
- D. A Big Build Dream Is To World
33. 若输入为 WorldYiwu AsiaShanghai ChinaHangzhou ZhejiangJinhua #, 则输出是 ( )。
- A. WorldYiwu AsiaShanghai ChinaHangzhou ZhejiangJinhua

- B.worldyiwu asiashanghai chinahangzhou zhejiangjinhua  
C.AsiaShanghai ChinaHangzhou WorldYiwu ZhejiangJinhua  
D.asiashanghai chinahangzhou worldyiwu zhejiangjinhua

34. (4分) 若输入为 I am 12 years old. #, 则输出是 ( )。
- A.am i old years                      B.am I old years  
C.I am years old.                      D.I am 12 years old.

### 三、完善程序 (单选题, 每小题 3 分, 共计 30 分)

- (1) 假设有一个无向图, 将这个无向图的某一条欧拉路或者欧拉回路用算法实现并打印出来。

输入格式:

第 1 行是  $n$  和  $m$ , 表示有  $n$  个点和  $m$  条边, 以下  $m$  行描述每条边连接的两点。

输入样例:

```
5 5
1 2
2 3
3 4
4 5
5 1
```

输出格式:

欧拉路或欧拉回路。

输出样例:

```
1 5 4 3 2 1
```

```
01 #include<bits/stdc++.h>
02 using namespace std;
03 #define MAXN 105
04 int g[MAXN][MAXN];
05 int du[MAXN];
06 int circuit[MAXN];
07 int n,e,circuitpos,x,y,start;
08 void find_circuit(int i)
09 {
10 for (int j = 1; j <= n; j++)
```

```
11 if (①)
12 {
13 g[j][i] = 0;
14 g[i][j] = 0;
15 find_circuit(j);
16 }
17 ②;
18 }
19 int main()
20 {
21 memset(g,0,sizeof(g));
22 cin >> n >> e;
23 for (int i = 1; i <= e; i++)
24 {
25 cin >> x >> y;
26 g[y][x] = 1;
27 ③;
28 du[x]++;
29 du[y]++;
30 }
31 start = 1;
32 for (int i = 1; i <= n; i++)
33 if (④)
34 start = i;
35 circuitpos = 0;
36 ⑤;
37 for (int i = 1; i <= circuitpos; i++)
38 cout << circuit[i] << ' ';
39 cout << endl;
40 return 0;
41 }
```

35. ①处应填 ( )。

A. `g[i][j] == 0`

B. `g[i][j] == 1`

C. `du[i] == 1`

D. `du[j] == 1`

36. ②处应填 ( )。

A. `circuit[++circuitpos] = i`

B. `circuit[circuitpos] = i`

C. `circuit[j] = i`

D. `circuit[i] = j`

37. ③处应填 ( )。

- A. start=1      B. start=0      C. g[x][y]=1      D. g[x][y]=0

38. ④处应填 ( )。

- A. g[i][j] % 2 == 0      B. g[i][j] % 2 == 1  
C. du[i] % 2 == 0      D. du[i] % 2 == 1

39. ⑤处应填 ( )。

- A. find\_circuit(0)      B. find\_circuit(start)  
C. find\_circuit(1)      D. find\_circuit(n)

(2) 设一棵  $n$  结点的二叉树  $tree$  的中序遍历为  $(1, 2, 3, \dots, n)$ , 其中数字  $1, 2, 3, \dots, n$  为结点编号。每个结点都有一个分数 (均为正整数), 记第  $i$  个结点的分数为  $d_i$ ,  $tree$  及它的每棵子树都有一个加分, 任意一棵子树  $subtree$  (也包含  $tree$  本身) 的加分计算方法如下:

$subtree$  的左子树的加分  $\times subtree$  的右子树的加分  $+ subtree$  的根的分数

若某棵子树为空, 规定其加分为 1。叶子的加分就是叶子结点本身的分数, 不考虑它的空子树。试求一棵符合中序遍历为  $(1, 2, 3, \dots, n)$  且加分最高的二叉树  $tree$ 。

要求输出: (1)  $tree$  的最高加分; (2)  $tree$  的前序遍历。

输入格式:

第 1 行是一个整数  $n$ , 为结点个数。第 2 行是  $n$  个用空格隔开的整数, 为每个结点的分数 ( $0 < \text{分数} < 100$ )。

输出格式:

第 1 行是一个整数, 为最高加分 (结果不会超过 `int` 范围)。第 2 行是  $n$  个用空格隔开的整数, 为该树的前序遍历。如果存在多种方案, 则输出字典序最小的方案。

数据范围:

$n < 30$ 。

输入样例:

5  
5 7 1 2 10

输出样例:

145  
3 1 2 4 5

```
01 #include<bits/stdc++.h>
02 using namespace std;
03 const int N=30;
04 int n, w[N], f[N][N], rt[N][N];
05 void print(int l, int r)
06 {
07 if(l > r)
08 return;
09 ①;
10 printf("%d ",root);
11 ②;
12 print(root+1, r);
13 }
14 int main()
15 {
16 memset(f,0,sizeof(f));
17 scanf("%d", &n);
18 for (int i=1; i<=n; ++i)
19 {
20 scanf("%d", &w[i]);
21 ③;
22 rt[i][i] = i;
23 }
24 for (int l=n; l>=1; l--)
25 for(int r=l+1; r<=n; r++)
26 for(int root=l; root<=r; root++)
27 {
28 int lw=f[l][root-1];
29 ④;
30 if(lw == 0)
31 lw = 1;
32 if(rw == 0)
33 rw = 1;
34 if(⑤)
35 {
36 f[l][r] = lw*rw+w[root];
37 rt[l][r] = root;
38 }
39 }
40 printf("%d\n", f[1][n]);
```

```
41 print(1, n);
42 printf("\n");
43 return 0;
44 }
```

40. ①处应填 ( )。

A. `int root = 0`

B. `int root = 1`

C. `int root = rt[l][r]`

D. `int root = rt[1][n]`

41. ②处应填 ( )。

A. `print(1, root-1)`

B. `print(l, root-1)`

C. `print(1, root)`

D. `print(l, root)`

42. ③处应填 ( )。

A. `f[i][i] = 0`

B. `f[i][i] = 1`

C. `f[i][i] = i`

D. `f[i][i] = w[i]`

43. ④处应填 ( )。

A. `int rw=f[root][r]`

B. `int rw=f[root+1][r]`

C. `int rw=f[root][n]`

D. `int rw=f[root+1][n]`

44. ⑤处应填 ( )。

A. `lw*rw+w[root] > f[l][r]`

B. `lw*rw+w[root] < f[l][r]`

C. `lw*rw+w[root] > f[1][n]`

D. `lw*rw+w[root] < f[1][n]`

## 普及组 CSP-J 2024 初赛模拟卷 9

扫码获取  
答案解析

## 一、单项选择题（共 15 题，每题 2 分，共计 30 分；每题有且仅有一个正确选项）

1. 已知数组 A 中，每个元素  $A[i][j]$  在存储时要占 3 字节，设  $i$  从 1 变化到 8， $j$  从 1 变化到 10，分配内存时是从地址 SA 开始连续按行存储分配的。试问： $A[4][7]$  的起始地址是什么？（ ）

A. SA+108      B. SA+141      C. SA+111      D. SA+126

2. 以下关于算法的描述正确的是（ ）。

A. 算法可以没有输出  
B. 算法至少有一个输入  
C. 算法必须在计算机上用某种语言实现  
D. 算法的改进在很大程度上推动了计算机科学与技术的进步

3. 执行下述 C++ 代码，输出的结果是（ ）。

```
#include<bits/stdc++.h>
using namespace std;
int main()
{
 int pos=2;
 string s("noip");
 cout<<s[++pos]<<endl;
 return 0;
}
```

A. o      B. i      C. p      D. 空格

4. （ ）表示只读存储器。

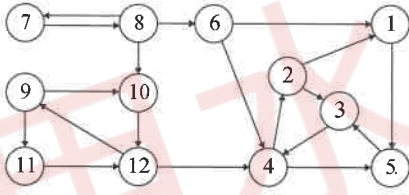
A. HDD      B. SSD      C. ROM      D. RAM

5. 有关下面 C++ 代码的说法，正确的是（ ）。

```
int cnt;
int fun(int a, int b)
```

```
{
 int tmp;
 if(0==b) tmp=a;
 else
 {
 tmp = fun(b,a%b);
 cnt++;
 }
 return tmp;
}
```

- A. fun 函数可以求出 a 和 b 的最大公共质因子  
B. fun 函数可能会死循环  
C. 如果 a 小于 9, 那么 cnt 的值不会超过 20  
D. fun 函数能够求出 a 和 b 的最小公倍数
6. 以下哪个不属于与 STL 有关的函数? ( )  
A. swap                  B. sort                  C. max                  D. freopen
7. 在顺序表(1, 3, 7, 10, 14, 15, 19, 26, 38, 47, 85)中, 用二分法查找 13, 所需的关键码比较的次数为 ( )。  
A. 2                      B. 3                      C. 4                      D. 5
8. 在下列各种排序算法中, 关键字比较的次数与记录的初始排列次序无关的是 ( )。  
A. 插入排序              B. 快速排序              C. 选择排序              D. 冒泡排序
9. 下列说法中正确的是 ( )。  
A. 计算机网络按照地理范围分为局域网、城域网和广域网  
B. 互联网的基础 TCP/IP 是七层协议  
C. 每台计算机配置的都是 C 类 IP 地址  
D. 中国的计算机 IP 地址已经全部升级到了 IPv6 地址
10. 在 U 盘中发现计算机病毒后, 较为彻底的清除方法是 ( )。  
A. 删除 U 盘文件                  B. 格式化 U 盘  
C. 用消毒水喷洒在 U 盘上          D. 用杀毒软件处理

11. 下列关于十进制数 101 的说法中错误的是 ( )。
- 注：这里 B 表示二进制 Binary，H 表示十六进制 Hexadecimal。
- A. 该数原码为 01100101 B      B. 该数反码为 65 H  
C. 该数真值为 01100011 B      D. 该数补码为 65 H
12. 动态规划将一个问题分解为一系列子问题来求解。下面关于子问题的描述中正确的是 ( )。
- A. 具有重叠子问题的性质  
B. 和分治法的子问题类似  
C. 不具有最优子结构的性质  
D. 问题的最优解可以由部分子问题的非最优解推导出来
13. 在下面的有向图中，有多少个强连通分量？ ( )
- 
- A. 3      B. 4      C. 5      D. 6
14. 从 4 台甲型电视机和 5 台乙型电视机中任意取出 3 台，其中至少要甲型电视机与乙型电视机各一台，则不同的取法共有 ( ) 种。
- A. 140      B. 70      C. 80      D. 35
15. 在 32 位计算机中，用补码表示的 C++ 的整型变量 int 能够表示的数据范围是 ( )。
- A.  $0 \sim 2^{31} - 1$       B.  $0 \sim 2^{32}$       C.  $-2^{31} \sim 2^{31} - 1$       D.  $-2^{31} + 1 \sim 2^{31}$

二、阅读程序（程序输入不超过数组或字符串定义的范围；判断题正确填√，错误填×；除特殊说明外，判断题每题 1.5 分，选择题每题 3 分，共计 40 分）

(1)

```
01 #include <bits/stdc++.h>
02 using namespace std;
03 priority_queue < int, vector<int>, greater<int> > q;
```

```
04 int main()
05 {
06 int n,x,ans=0,t1=0,t2=0,t3=0;
07 cin >> n;
08 for(int i=0; i<n; i++)
09 {
10 cin>>x;
11 q.push(x);
12 }
13 for(int i=n; i>1; i--)
14 {
15 t1 = q.top();
16 q.pop();
17 t2 = q.top();
18 q.pop();
19 t3 = t1+t2;
20 ans += t3;
21 q.push(t3);
22 }
23 cout<<ans<<endl;
24 return 0;
25 }
```

#### ■ 判断题

16. priority\_queue 是 STL 中的优先队列。 ( )
17. 将第 3 行中的 greater<int>去掉, 不管输入如何, 程序的运行结果都不变。 ( )
18. 将第 15 行和第 17 行互换, 不管输入如何, 程序的运行结果都不变。 ( )
19. 将第 21 行去掉, 不管输入如何, 程序的运行结果都不变。 ( )

#### ■ 选择题

20. 若输入 3 1 2 9, 则输出 ans 为 ( )。
- A. 12                      B. 22                      C. 23                      D. 15
21. 若输入 4 5 4 3 6, 则输出 ans 为 ( )。
- A. 30                      B. 36                      C. 37                      D. 38

(2)

```
01 #include <bits/stdc++.h>
02 using namespace std;
03 typedef long long LL;
04 LL fun(LL a, LL b, LL &x, LL &y)
05 {
06 LL t;
07 if(0==b)
08 {
09 x=1;
10 y=0;
11 return a;
12 }
13 LL d = fun(b,a%b,x,y);
14 t=x;
15 x=y;
16 y=t-(a/b)*y;
17 return d;
18 }
19 int main()
20 {
21 LL a,b,x=0,y=0;
22 cin >> a >> b;
23 cout << fun(a, b, x, y) << endl;
24 cout << x << " " << y << endl;
25 return 0;
26 }
```

## ■ 判断题

22. 将第 3 行中的 `typedef long long LL` 改为 `#define long long LL`, 程序的运行结果不会改变。 ( )
23. 将第 4 行中的两个 `&` 去掉, 程序的运行结果不会改变。 ( )
24. 若输入  $b=0$ , 则输出结果是  $a$   
 $0 \ 1$ 。 ( )
25. 若输入两个为素数的正整数, 则输出结果中,  $x$  和  $y$  一定有负数。 ( )

## ■ 选择题

26. 若输入为 0 2024, 则输出为 ( )。

A. 2024

B. 2024

C. 0

D. 0

1 1

0 1

1 1

0 1

27. 若输入为 36 48, 则输出为 ( )。

A. 144

B. 144

C. 12

D. 12

1 -1

-1 1

1 -1

-1 1

(3)

```
01 #include <bits/stdc++.h>
02 using namespace std;
03 int a[101][3], b[101];
04 void dfs(int k, int w)
05 {
06 if(1==w)
07 cout<<k<<" ";
08 if(a[k][1]!=0)
09 dfs(a[k][1],w);
10 if(2==w)
11 cout<<k<<" ";
12 if(a[k][2]!=0)
13 dfs(a[k][2],w);
14 if(3==w)
15 cout<<k<<" ";
16 }
17 int main()
18 {
19 int n, m, t, root;
20 cin>>n;
21 for (int i=1; i<=n; i++)
22 {
23 cin>>t;
24 cin>>a[t][1]>>a[t][2];
25 b[a[t][1]]=1;
26 b[a[t][2]]=1;
27 }
28 for (int i=1; i<=n; i++)
```

```
29 if(b[i]==0)
30 {
31 root=i;
32 break;
33 }
34 dfs(root,1);
35 cout<<endl;
36 return 0;
37 }
```

### ■ 判断题

28. dfs 函数是处理二叉树的前、中、后序遍历的。 ( )
29. 若将第 24 行 `cin>>a[t][1]>>a[t][2]` 更改为 `cin>>a[1][t]>>a[2][t]`, 程序的运行结果不会改变。 ( )
30. 若将第 29 行去掉, 程序的运行结果不会改变。 ( )
31. 若将第 34 行 `dfs(root,1)` 更换为 `dfs(root,4)`, 程序的运行结果不会改变。 ( )

### ■ 选择题

32. 若输入如下数据, 则输出是 ( )。

```
8
1 2 4
2 0 0
4 8 0
3 1 5
5 6 0
6 0 7
8 0 0
7 0 0
```

- A. 3 1 2 4 8 5 6 7  
C. 2 8 4 1 7 6 5 3

- B. 2 1 8 4 3 6 7 5  
D. 3 2 1 8 4 5 7 6

33. 若输入和第 32 题一样, 将第 34 行 `dfs(root,1)` 更换为 `dfs(root,2)`, 则输出是 ( )。

- A. 3 1 2 4 8 5 6 7  
C. 2 8 4 1 7 6 5 3

- B. 2 1 8 4 3 6 7 5  
D. 3 2 1 8 4 5 7 6

34. (4 分) 若输入和第 32 题一样, 将第 34 行 `dfs(root,1)` 更换为 `dfs(root,3)`, 则输出是 ( )。

A. 3 1 2 4 8 5 6 7

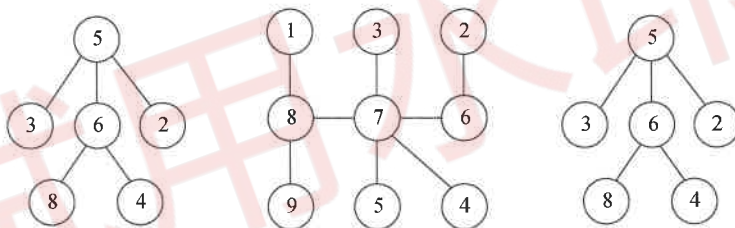
B. 2 1 8 4 3 6 7 5

C. 2 8 4 1 7 6 5 3

D. 3 2 1 8 4 5 7 6

### 三、完善程序 (单选题, 每小题 3 分, 共计 30 分)

(1) 米老鼠上次在唐老鸭的迷宫城堡里玩了很久, 现在她也想设计一个迷宫让唐老鸭来走。但是她设计迷宫的思路不一样, 她认为所有的通道都应该是双向连通的, 也就是说, 如果有一个通道连通了房间 A 和房间 B, 那么既可以通过它从房间 A 走到房间 B, 也可以通过它从房间 B 走到房间 A。为了提高难度, 米老鼠希望任意两个房间有且仅有一条路径可以相通 (除非走了回头路)。米老鼠现在把她的设计图给你, 让你帮忙判断她的设计图是否符合她的设计思路。比如下面的例子, 前两个是符合条件的, 但是最后一个有两种方法从 5 到达 8。



输入格式:

输入包含多组数据, 每组数据是一个以 0 0 结尾的整数对列表, 表示了一条通道连接的两个房间的编号。房间的编号至少为 1, 且不超过 100 000。每两组数据之间有一个空行。整个输入以两个 -1 结尾。

输出格式:

对于输入的每一组数据, 输出仅包括一行。如果该迷宫符合米老鼠的思路, 那么输出 YES, 否则输出 NO。

输入样例:

```
6 8 5 3 5 2 6 4 5 6 0 0
8 1 7 3 6 2 8 9 7 5 7 4 7 8 7 6 0 0
3 8 6 8 6 4 5 3 5 6 5 2 0 0
-1 -1
```

输出样例:

YES

YES

NO

```
01 #include<iostream>
02 using namespace std;
03 #define MAXN 100005
04 int fa[MAXN];
05 int visit[MAXN];
06 int path, node;
07 void init(int n){
08 for(int i=1;i<=n;++i) {
09 fa[i] = i;
10 visit[i] = 0;
11 }
12 path = 0;
13 node = 0;
14 }
15 int get(int x){
16 if(x == fa[x])
17 return x;
18 else
19 return ①;
20 }
21 int main(){
22 int a,b;
23 bool flag;
24 cin>>a>>b;
25 while(a!=-1 && b!=-1){
26 init(MAXN);
27 flag = true;
28 if(0==a && 0==b)
29 path--;
30 while(②){
31 path++;
32 if(visit[a]==0){
33 visit[a]=1;
34 node++;
35 }
```







## 普及组 CSP-J 2024 初赛模拟卷 10

扫码获取  
答案解析

## 一、单项选择题（共 15 题，每题 2 分，共计 30 分；每题有且仅有一个正确选项）

1. 用 C++ 语言将一个大写字母转换为对应的小写字母（比如将大写字母 A 转换为小写字母 a），以下哪个代码不正确？（ ）

A. `c+=32`      B. `c-=32`      C. `c^=' '`      D. `c^=32`

2. 下列说法中错误的是（ ）。

A. 栈、队列、树和图都属于数据结构的范畴  
B. CSP-J/CSP-S 是中国计算机学会举办的程序设计竞赛  
C. 背包问题属于动态规划算法中的一种类型  
D. 按照分布的地理范围，计算机网络类型可以分为星形、环形和总线型

3. 下列 C++ 数组的定义中，会丢失数据的是（ ）。

A. `char str1[] = {'C', 'S', 'P'};`  
B. `int num1[] = {3, 6, 9};`  
C. `char str2[] = {'CSP-J', 'CSP-S', 'NOIP'};`  
D. `float num2[] = {1.0, 3.0, 5.0};`

4. 执行下面的 C++ 代码，输出是（ ）。

```
#include<bits/stdc++.h>
using namespace std;
int main()
{
 int tmp = 1;
 for(int i=1; i<10; i++)
 for(int j=1; j<5; j++)
 if(i%j == 1)
 tmp++;
 cout<<tmp;
 return 0;
}
```

A. 10      B. 11      C. 12      D. 13

5. 24 位真彩色能表示 ( ) 种颜色。  
A. 16 777 216      B. 256      C. 24      D. 65 536
6. 以下哪个不属于 STL 中队列 (queue) 的操作函数? ( )  
A. push      B. pop      C. empty      D. top
7. 归并排序算法的空间复杂度是 ( )。  
A.  $O(\log n)$       B.  $O(n)$       C.  $O(1)$       D.  $O(n \log n)$
8. 下列 C++ 代码执行后不能输出 "NOIP" 的是 ( )。  
A. `string s("NOIP"); cout<<s<<endl;`  
B. `string s="NOIP"; cout<<s<<endl;`  
C. `string s("NOIP"); cout<<s[1]<<s[2]<<s[3]<<s[4]<<endl;`  
D. `string s{"NOIP"}; cout<<s<<endl;`
9. 哈夫曼树的构造用到了 ( ) 的算法思想。  
A. 二分查找      B. 搜索回溯      C. 贪心      D. 动态规划
10. 在 C++ 语言中, 一个字符指针变量 `char *p` 占 ( ) 字节。  
A. 1      B. 2      C. 8      D. 4
11. 某数列有 2024 个各不相同的单元, 由低至高按序排列。现要对该数列进行二分搜索 (binary search)。在最坏情况下, 需搜索 ( ) 个单元。  
A. 2024      B. 11      C. 12      D. 1012
12. 在 C++ 程序中, 判断 `a` 不等于 0 且 `b` 不等于 0 且 `c` 不等于 0 的正确的条件表达式是 ( )。  
A. `!((a!=0) || (b!=0) || (c!=0))`  
B. `!((a!=0) && (b!=0) && (c!=0))`  
C. `a && b && c`  
D. `(a=0) && (b=0) && (c=0)`
13. 某二叉树有 4 个结点, 则该二叉树的不同形态有 ( ) 种。  
A. 12      B. 14      C. 10      D. 16

14. 设含有 8 个元素的集合的全部子集数为  $S$ ，其中由 6 个元素组成的子集数为  $T$ ，则  $T/S$  的值为 ( )。
- A.  $7/16$                       B.  $7/64$                       C.  $7/32$                       D.  $21/256$
15. 对于给定的序列  $\{a_k\}$ ，我们把  $(i, j)$  称为逆序对，当且仅当  $i < j$  且  $a_i > a_j$ 。那么序列 1, 9, 2, 4, 8, 6 的逆序对有 ( ) 个。
- A. 7                              B. 6                              C. 5                              D. 4

二、阅读程序（程序输入不超过数组或字符串定义的范围；判断题正确填√，错误填×；除特殊说明外，判断题每题 1.5 分，选择题每题 3 分，共计 40 分）

(1)

```
01 #include<bits/stdc++.h>
02 using namespace std;
03 int a[10];
04 void print_num(){
05 cout<<'1';
06 for(int i=2;i<=9;++i){
07 if(a[i]==0)
08 cout<<'+';
09 else
10 cout<<'-' ;
11 cout<<i;
12 }
13 cout<<"=9"<<endl;
14 return;
15 }
16 int main(){
17 for(int i=0; i<=255; i++){
18 int k=i, s = 1;
19 for(int j=9;j>=2;j--){
20 if(k&1){
21 s-=j;
22 a[j]=1;
23 }
24 else{
25 s+=j;
26 a[j]=0;
```

```
27 }
28 k>>=1;
29 }
30 if(s==9)
31 print_num();
32 }
33 return 0;
34 }
```

#### ■ 判断题

16. 将第 8 行和第 10 行交换, 程序的运行结果不会改变。 ( )
17. 将第 8 行中的 '+' 改为 "+", 程序的运行结果不会改变。 ( )
18. 将第 20 行中的  $k\&1$  改为  $k\&2$ , 程序的运行结果不会改变。 ( )
19. 将第 28 行中的  $k>>=1$  改为  $k/=2$ , 程序的运行结果不会改变。 ( )

#### ■ 选择题

20. 输出数据共有 ( ) 行。  
A. 11                      B. 10                      C. 12                      D. 13
21. 程序主要用到了 ( ) 的算法思想。  
A. 贪心                      B. 二分                      C. 二进制                      D. 动态规划

(2)

```
01 #include<bits/stdc++.h>
02 using namespace std;
03 stack <char> s;
04 int main()
05 {
06 string a;
07 int len,mid,next,top;
08 getline(cin,a);
09 len=a.length();
10 mid=len/2-1;
11 for(int i=0; i<=mid; i++)
12 s.push(a[i]);
13 if(len%2==0)
14 next=mid+1;
15 else
```

```
16 next=mid+2;
17 for(int i=next;i<=len-1;i++)
18 {
19 if(a[i] != s.top())
20 break;
21 s.pop();
22 }
23 if(s.size()==0)
24 cout<<"yes";
25 else
26 cout<<"no";
27 return 0;
28 }
```

### ■ 判断题

22. 将第 6 行中的 `string a` 改为 `char a[256]`, 程序的输出结果不变。 ( )
23. 即使输入的字符串中间有空格, `getline` 函数也支持相应的处理。 ( )
24. 将第 11 行中的 `int i=0` 改为 `int i=1`, 程序的输出结果不变。 ( )
25. 若输入 `abc cba`, 同时将第 8 行的 `getline(cin,a)` 改为 `cin>>a`, 程序的输出结果为 `no`。 ( )

### ■ 选择题

26. 若输出结果为 `yes`, 则输入可能为 ( )。
- A. `abc big big abc`                      B. `abc big big cba`  
C. `abc big gib abc`                      D. `abc big gib cba`
27. 若输出结果为 `no`, 则输入可能为 ( )。
- A. `1234567890987654321`                  B. `12345 6789899876 54321`  
C. `12345 678909876 54321`                D. `12345 6789009876 54321`

### (3)

```
01 #include<bits/stdc++.h>
02 using namespace std;
03 struct node
04 {
05 int data;
06 node *next;
```

```
07 };
08 int n,m;
09 node *head,*p,*r;
10 int main()
11 {
12 cin >> n >> m;
13 head = new node;
14 head->data = 1;
15 head->next = NULL;
16 r = head;
17 for(int i=2;i<=n;i++)
18 {
19 p = new node;
20 p->data = i;
21 p->next = NULL;
22 r->next = p;
23 r = p;
24 }
25 r->next = head;
26 r = head;
27 for(int i=1;i<=n;i++)
28 {
29 for(int j=1;j<=m-2;j++)
30 r = r->next;
31 cout<<r->next->data<<" ";
32 r->next = r->next->next;
33 r = r->next;
34 }
35 return 0;
36 }
```

#### ■ 判断题

28. node 这个结构体在内存空间中占 8 字节。 ( )
29. 程序中有 new 却没有做 delete, 输入  $n$  大于 10 必然会导致程序运行时崩溃。 ( )
30. 若输入为 8 5, 去掉第 15 行, 程序的运行结果不会改变。 ( )
31. 本程序用到了双向链表的技巧。 ( )

## ■ 选择题

32. 若输入为 8 5, 则输出是 ( )。
- A. 5 2 7 8 1 4 6 3                      B. 5 2 8 7 1 4 6 3  
C. 5 2 8 7 4 1 6 3                      D. 5 2 8 7 1 4 3 6
33. 若输入为 11 4, 则输出是 ( )。
- A. 4 8 1 6 11 7 3 2 5 10 9              B. 4 8 6 1 11 7 3 2 5 10 9  
C. 4 8 1 11 6 7 3 2 5 10 9              D. 4 8 1 6 11 7 2 3 5 10 9
34. (4 分) 程序运行的时间复杂度是 ( )。
- A.  $O(n^2)$                       B.  $O(\log n)$                       C.  $O(nm)$                       D.  $O(m)$

## 三、完善程序 (单选题, 每小题 3 分, 共计 30 分)

- (1) 交叉字符串问题。给出 3 个字符串  $s_1$ 、 $s_2$  和  $s_3$ , 判断  $s_3$  是否可以由  $s_1$  和  $s_2$  两个字符串经过交叉组合而成, 组合过程不能改变字符在  $s_1$  和  $s_2$  中的原本顺序。如  $s_1 = \text{"aabcc"}$ ,  $s_2 = \text{"dbbca"}$ ,  $s_3 = \text{"aadbcbcbac"}$  或  $s_3 = \text{"aadbbaaccc"}$ 。给了两种  $s_3$  的情况, 第一种情况下答案是 yes, 第二种情况下答案是 no, 因为找不到任何一种  $s_1$  和  $s_2$  的交叉组合方式可以组合成  $s_3$ 。

输入格式:

第 1 行输入一个整数  $N$ 。第 2 行到第  $N+1$  行输入 3 个字符串对  $s_1, s_2, s_3$ ,  $s_1$  和  $s_2$  的长度都不超过 200,  $s_3$  的长度等于  $s_1$  和  $s_2$  的长度之和。

输出格式:

每个测试样例输出一行, yes 代表可以构成, no 代表不能。

输入样例:

```
3
cat tree tcraete
cat tree catrtee
cat tree cttaree
```

输出样例:

```
yes
yes
No
```

```
01 #include <bits/stdc++.h>
02 using namespace std;
03 string str1,str2,str;
04 bool pos;
05 int vis[205][205];
06 void dfs(int s1,int s2,int s){
07 if(s1==str1.length() && s2==str2.length()){
08 ①;
09 return;
10 }
11 if(②)
12 return;
13 if(③)
14 return;
15 vis[s1][s2]=1;
16 if(str1[s1]==str[s])
17 ④;
18 if(str2[s2]==str[s])
19 dfs(s1,s2+1,s+1);
20 }
21 int main()
22 {
23 int t,n;
24 cin>>t;
25 while(t--){
26 cin>>str1>>str2>>str;
27 ⑤;
28 memset(vis,0,sizeof(vis));
29 dfs(0,0,0);
30 if(pos)
31 cout<<"yes"<<endl;
32 else
33 cout<<"no"<<endl;
34 }
35 return 0;
36 }
```

35. ①处应填 ( )。

A. pos=true

B. pos=false

C. vis[s1][s2]=1

D. vis[s1][s2]=0

36. ②处应填 ( )。

- A. `str1[s1]==str[s] && str2[s2]!=str[s]`
- B. `str1[s1]!=str[s] && str2[s2]!=str[s]`
- C. `str1[s1]!=str[s] && str2[s2]==str[s]`
- D. `str1[s1]==str[s] && str2[s2]==str[s]`

37. ③处应填 ( )。

- A. `vis[s1][s2]`
- B. `vis[s1][s2]==0`
- C. `pos==true`
- D. `pos==false`

38. ④处应填 ( )。

- A. `dfs(s1,s2,s+1)`
- B. `dfs(s1+1,s2+1,s)`
- C. `dfs(s1+1,s2,s+1)`
- D. `dfs(s1+1,s2+1,s+1)`

39. ⑤处应填 ( )。

- A. `vis[s1][s2]==1`
- B. `vis[s1][s2]==0`
- C. `pos==true`
- D. `pos==false`

(2) 设有  $N$  堆沙子排成一排, 编号为  $1, 2, \dots, N$  ( $N$  小于 1000), 每堆沙子有一定的数量, 用  $a[k]$  表示第  $K$  堆沙子的数量值。现在要将  $N$  堆沙子归并成为一堆, 归并的过程为每次只能将相邻的两堆沙子堆成一堆, 合并后的这堆沙子的代价为这两堆沙子的数值和, 这样经过  $N-1$  次归并之后, 最后成为一堆。不同的归并方案的总代价值是不同的。现给出  $N$  堆沙子的数量后, 请找出一种合理的归并方法, 使总的归并代价最小。  
输入格式:

输入有  $n+1$  行, 第 1 行为  $n$  的值, 从第 2 行开始, 为  $n$  个正整数, 表示各堆沙子的数量值。

输出格式:

只有一行, 表示最小的总代价, 结果保证小于  $2^{31}$ 。

输入样例:

10  
12  
3  
13  
7

8  
23  
14  
6  
9  
34

输出样例:

398

```
01 #include <bits/stdc++.h>
02 using namespace std;
03 int main()
04 {
05 int a[105],s[105],f[100][100];
06 int j,n,p,m;
07 cin>>n;
08 s[0]=0;
09 for(int i=1;i<=n;i++)
10 {
11 cin>>a[i];
12 ①;
13 f[i][i]=0;
14 }
15 for(p=1;p<=n-1;p++)
16 for(int i=1;i<n;i++)
17 {
18 ②;
19 if(j<=n)
20 {
21 m=INT_MAX;
22 for(int k=i;k<j;k++)
23 m=min(m, ③);
24 ④;
25 }
26 }
27 cout<<⑤;
28 return 0;
29 }
```

40. ①处应填 ( )。

A.  $s[i]=s[i-1]+a[i-1]$

B.  $s[i]=s[i-1]+a[i]$

C.  $s[i]=s[i-1]+a[i+1]$

D.  $s[i]=\min(s[i-1],a[i])$

41. ②处应填 ( )。

A.  $j=i+p$

B.  $j=i+p-1$

C.  $j=i+p+1$

D.  $j=\max(i,p)$

42. ③处应填 ( )。

A.  $f[i][k] + f[k+1][j]$

B.  $f[i][k] + f[k][j]$

C.  $f[i][k-1] + f[k+1][j]$

D.  $f[i][k+1] + f[k+1][j]$

43. ④处应填 ( )。

A.  $f[i][j] = m + s[j] - s[i]$

B.  $f[i][j] = m + s[j] - s[i-1]$

C.  $f[i][j] = m + s[j-1] - s[i]$

D.  $f[i][j] = m + s[j-1] - s[i-1]$

44. ⑤处应填 ( )。

A.  $f[0][n]$

B.  $f[0][n-1]$

C.  $f[1][n]$

D.  $f[1][n-1]$